

- reduces spatial size of convolved features
- decrease the computational power req to process the data
- extracts dominant features which are rotational and positional invariant

(iv) Pooling

→ after extracting the region, what to do?

→ types: (i) max pooling → sharpening the area: (for age detection)

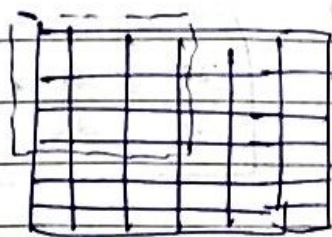
(ii) average pooling → removing noise by removing non-dominant features
 ↑ Smoothing the values/area
 → remove noise by averaging

(v) Flattening of pooling layer



input to dense layer → decides what will be the work?

(vi) CNN - sparsely connected MLP



no dense connection

On applying filter, # sparse !!

→ 9 nos. selected from first 3 rows

36 values → all nos. not considered (9/36)

NOTE: Same filter matrix applied !!

(after convolution → we can have activation



pooling

↓ Flatten !!

Day 15 (23/02/26)

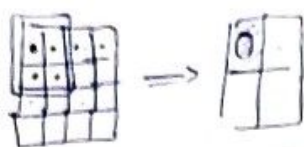
• Backpropagation in CNN

(i) Parameters — filters

weight matrix of the dense layer

→ MLP

• how to backprop from pooling output (!)



$$\frac{\partial E}{\partial C} = \frac{\partial E}{\partial P} \cdot \frac{\partial P}{\partial C}$$

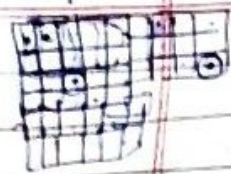
classmate
Date _____
Page _____

NOTE: When we are differentiating
wrt some quantity $\Rightarrow$ we need

$$\frac{\partial E}{\partial F} = \frac{\partial S}{\partial F} \times \frac{\partial P}{\partial C} \times \frac{\partial E}{\partial P}$$

filter convolution

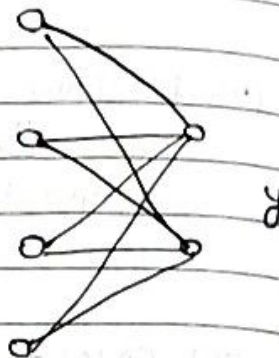
to check $\Rightarrow$ which quantity is fully dependent on it!!



$$\begin{cases} E = f(y) \\ y = f(z) \\ z = f(p) \end{cases} \quad \frac{\partial E}{\partial p} = \frac{\partial z}{\partial p} \frac{\partial y}{\partial z} \frac{\partial E}{\partial y}$$

(done by all P_{ij}) for the MLP part

Pooling



$$\frac{\partial P}{\partial C} = \frac{\partial (\max(C_{00}, C_{01}, C_{10}, S_{11}))}{\partial C}$$

(for all C_{ij} we need to do)

$$\frac{\partial E}{\partial C} = \frac{\partial P}{\partial C} \times \frac{\partial E}{\partial P}$$

also for all i, j of F , consider k, l for C

Check on

$$\frac{\partial E}{\partial P} = \frac{\partial E}{\partial C} \times \frac{\partial C}{\partial P}$$

$$\text{or } \frac{\partial E}{\partial x} = \frac{\partial C}{\partial x} \times \frac{\partial E}{\partial C}$$

filter size

convolution

size etc

$$\frac{\partial E}{\partial F_{ij}} = \sum_{k=0}^n \frac{\partial E}{\partial C_{kl}} \times \frac{\partial C_{kl}}{\partial F_{ij}} = \text{Convolution} \left(A, \frac{\partial E}{\partial C} \right)$$

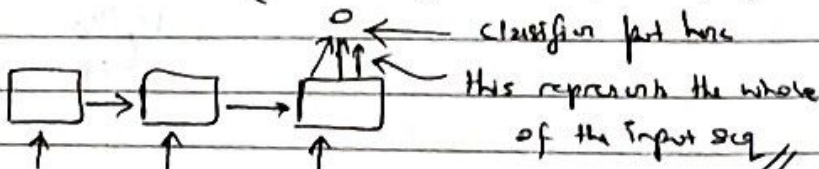
(do it with for F_{ij})

def/

Encoder - Decoder Model (Seq 2 Seq)

Calculation!!

$\rightarrow$ RNN Structure (Text classification), Pos (Seq $\rightarrow$ Seq)



$\rightarrow$ encoder: generation of embedding of input

produces intermediate representation {single fixed vector}

decoder: take intermediate representation

also there is a trigger - special token

this then output the result

as in it generates a new Seq conditioned on the encoded representation

works normally

works in reverse order

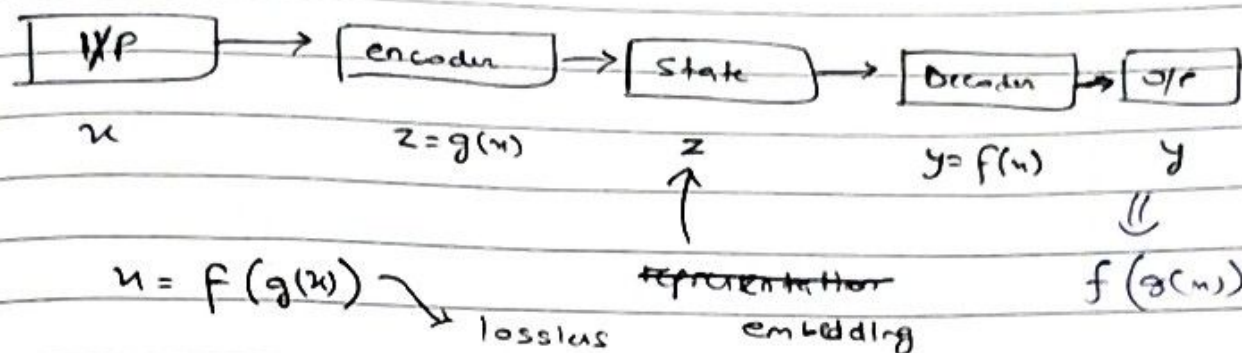
it is generative!!

for vanilla encoder-decoder

$\rightarrow$ Info gets lost
 $\rightarrow$ performance degrades
 $\rightarrow$ gradients struggle

$\Rightarrow$ that's why attention!!

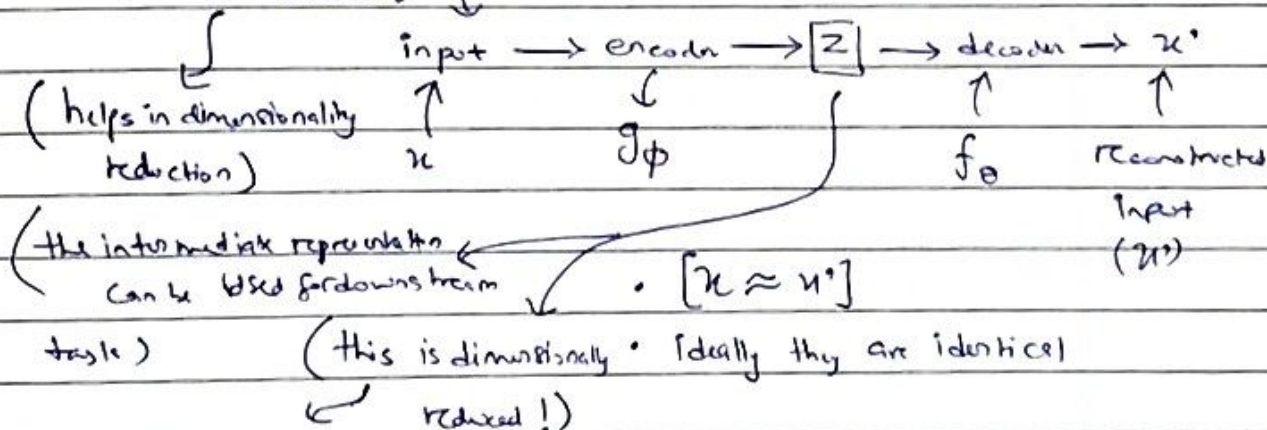
- No difference in the length of input and output seq
- NOTE: Encoder and decoder both are independent models
- # words in encoder may not equal # words in decoder



- NOTE: Encoder-decoder used for representation !! or end-to-end
- U-Net - popular CNN embedding representation (task)
- homogeneous: same models in both encoder and decoder
- heterogeneous: different models in " " "

(dimensionality reduction)

NOTE: Auto encoder



fed to different model!

→ downstream task

NOTE: encoder - decoder is an architecture pattern
↳ it is a framework (not a specific model)

Attention!!

Mid Sem

i) Representation Issues: (i) need of domain knowledge



(ii) expert bias

Can we learn it automatically from the dataset?

⇒ embedding - is the method to learn representation vector of the samples in the low dimensional space from the data



low dimensional vector: Retains the semantic meaning of representation the data in high dimensional space

ii) Issues with document-term representations

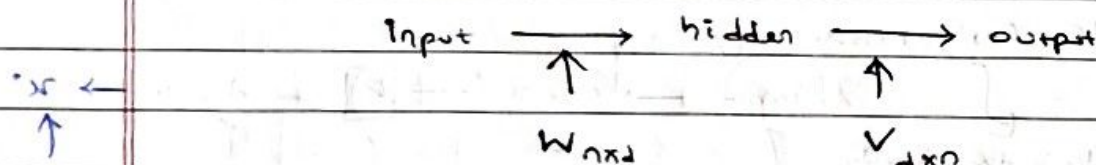
a) Large feature set

b) Many of the features may not be useful

c) sparse

d) Curse of dimensionality

iii) NOTE: in CBOW,



- either W or V can be taken as word's representation
- even take the average

Day 16 (26/02/26)

Parameter Estimation

- ↓ requires probabilistic model //
- (i) Maximum Likelihood Estimation (MLE) — When data is complete
 - (ii) Expectation Maximization (EM) — Can be used in both

MLE

- i) let's say we have dataset D ,
- ii) We know its ^{underlying} distribution — Could be binomial, gaussian
- iii) parameters for that distribution — we want to estimate //

MLE objective

We know D ,
but we
don't know θ

$$D = \{x_1, x_2, \dots, x_n\}$$

θ : parameters for generating D

so we find θ

from

$$P(x_i | \theta)$$

drawn from distribution

iid

$$L(D | \theta) = P(D | \theta) = P(x_1, x_2, \dots, x_n | \theta)$$

$$\Rightarrow L(D | \theta) = \prod_{i=1}^n P(x_i | \theta)$$

since all
samples
independent
to each
other

Apply log likelihood → to simplify estimates

$$\Rightarrow \log L(D | \theta) = \ell(D | \theta)$$

$$= \log \prod_{i=1}^n P(x_i | \theta)$$

→ Take derivative wrt θ

$$\frac{\partial \ell(D | \theta)}{\partial \theta} = 0$$

example: Tossing coin

$$D = \{H \ T \ H \ T \ \dots \ H\}$$

$$\approx \{1 \ 0 \ 1 \ 0 \ \dots \ 1\}$$

$$\approx \{x_1, x_2, x_3, \dots, x_n\}$$

→ Bernoulli's distribution

What are my parameters?

↑

$$P(H), P(T)$$

$$P(x_i | \theta) = \binom{1}{x_i} \theta^{x_i} (1-\theta)^{1-x_i}$$

θ we don't know

$$\text{Since } x_i \text{ is } 1 \text{ or } 0 \Rightarrow \binom{1}{x_i} = 1 //$$

$$\approx \theta^{(x_i)} (1-\theta)^{1-x_i}$$

$$L(D|\theta) = \prod_{i=1}^n P(x_i | \theta)$$

$$= \prod_{i=1}^n \theta^{x_i} (1-\theta)^{(1-x_i)}$$

$$= \theta^{\sum_{i=1}^n x_i} (1-\theta)^{\sum_{i=1}^n (1-x_i)}$$

Apply log likelihood,

$$L(D|\theta) = \sum_{i=1}^n x_i \ln \theta + \sum_{i=1}^n (1-x_i) \ln(1-\theta)$$

this tells us no. of 1s

(let's say k)

take derivative wrt θ

$$\frac{\partial L}{\partial \theta} = \frac{k}{\theta} + \frac{n-k}{(1-\theta)} (-1) = 0$$

$$\Rightarrow \frac{k}{\theta} - \frac{n-k}{(1-\theta)} = 0$$

$$\frac{k}{(1-\theta)(\theta)} + \frac{k}{(1-\theta)(\theta)} - \frac{n-k}{(1-\theta)(\theta)} = 0$$

$$\theta = \frac{k}{n}$$

$$\frac{k - \theta k - n\theta + \theta k}{\theta(1-\theta)}$$

$$k - n\theta = 0$$

What if the dataset follows gaussian distribution,
what is the θ ?

It's mean, variance i.e. $\theta = \{\mu, \sigma\}$

$$P(x_i | \theta) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{1}{2}\left(\frac{x_i - \mu}{\sigma}\right)^2}$$

$$L(D|\theta) = \prod_{i=1}^n P(x_i | \theta)$$

NOTE: assumption
is: all
samples are
independent

$$\ell(D|\theta) = \sum_{i=1}^n \ln P(x_i | \theta)$$

$$= \sum_{i=1}^n \ln \left(\frac{1}{\sqrt{2\pi}} \cdot \frac{1}{\sigma} \cdot e^{-\frac{1}{2}\left(\frac{x_i - \mu}{\sigma}\right)^2} \right)$$

$$= \sum_{i=1}^n \ln \frac{1}{\sqrt{2\pi}} + \sum_{i=1}^n \ln \frac{1}{\sigma} + \sum_{i=1}^n \left(-\frac{1}{2} \left(\frac{x_i - \mu}{\sigma} \right)^2 \right)$$

* ML practice view: Define Loss

Compute gradients

update weights //

classmate

Date
Page

Take partial derivative wrt σ and μ ,

$$\frac{\partial L}{\partial \mu} = + \frac{1}{2\sigma^2} \sum_{i=1}^n (2(\mu_i - \mu))(\mu_i - \mu) = 0$$

$$\sum_{i=1}^n \mu_i - \sum_{i=1}^n \mu = 0$$

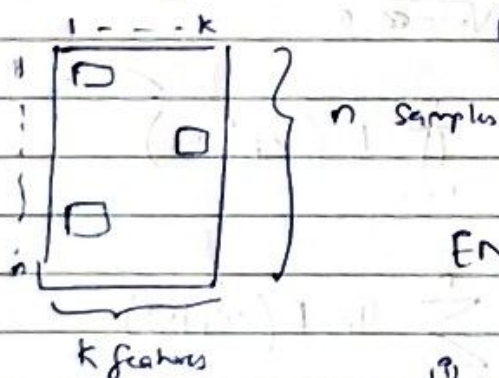
$$\Rightarrow \sum_{i=1}^n \mu_i = n\mu$$

$$\Rightarrow \mu = \frac{\sum_{i=1}^n \mu_i}{n} \rightarrow \text{this is Sample mean}$$

Similarly for $\frac{\partial L}{\partial \sigma}$ do!

EM

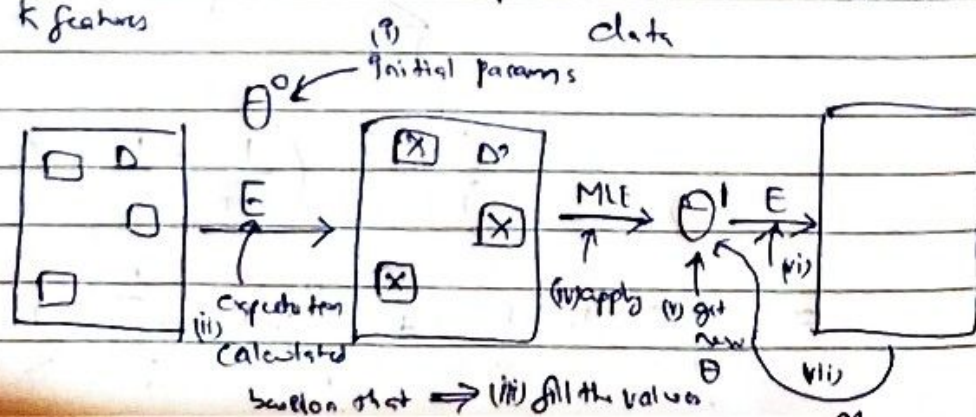
applied for both complete and incomplete dataset



- Objective
- (i) I want to predict θ
 - (ii) pls fill the empty cells

EM - some expected value producing considering initial data

$\{(D' - D) \text{ is the error!!}\}$



* probabilistic view: assumed distribution with parameters θ
 maximum likelihood $P(x|\theta)$ classmate
 equivalent to minimizing negative log likelihood Date
 update θ

What are the convergence?

- (i) when θ becomes same
- (ii) when points become same

(A)

given data $\rightarrow$ produce expected data

here we don't know the distribution

$\downarrow$
 apply MLE on it

$\downarrow$
 get new distribution/parameters

$\downarrow$
 Based on that again produce new data

$\uparrow$
 keep repeating till

(A) observed //

but if we know the distribution then

we take the probability of observing the data given the params

(but in both θ updated which are weights)

then MLE is the error function

NOTE: both cases are very different:

- (i) probabilistic view
- (ii) ML practice view

NOTE: It follows:

(i) start with initial parameters θ_{old}

E-step (ii) Compute the expected hidden information

$\rightarrow$ Expected hidden variable given X and θ_{old}

M-step (iii) use those expectations to perform MLE and obtain new parameters θ_{new}

(iv) Repeat until convergence

check memorials on it!!

Day 17 (11/03/20)

(i) Vanishing grad } problems in RNN
(ii) exploding grad }
→ suffers from Long Term
Depending

Short term memory

in RNN - 1 hidden state

in LSTM - 2 states - hidden state

cell state - long memory

→ stores global memory

remembers all global useful informations

from $t=0$ to current

note: h_t in RNN is similar to C_t in LSTM //

how much
info
to pass

forget gate - from global memory (~~C_{t-1}~~ C_{t-1})
input gate - from current info
output gate - from ^{new} C_t output //

Note: We is nothing but internal
information

of next hidden state

→ tells how much of C_t is useful //

- σ used as 0 to 1 (kind of gate)

for normalization

(short of
switch)

- tanh $\rightarrow (-1 \text{ to } 1)$!! - (normalize activation and maintain
value) !.

How to do ~~it~~ ~~its~~ computation (matrix format)

→ req gates and internal information ($\tilde{C}_t$)

note: In gru also, because of the gates, problems of RNN
is resolved!!

LSTM

- a) Internal information
↳ output from hidden layer

$$\tilde{C}_t = \tanh(W_c \cdot x_t + U_c h_{t-1} + b_c)$$

- b) Forget Gate - how much to throw away from C_{t-1} based on prev hidden state and present input

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$$

- c) Input Gate - what info from internal state $\tilde{C}_t$ we are going to keep

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$$

- d) output gate - what info from new cell state C_t we should carry forward to the next time step

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$$

↳ useful info from C_{t-1} and current

- e) new cell state: input becomes new cell state

$$C_t = C_{t-1} \otimes f_t \oplus i_t \otimes \tilde{C}_t$$

- f) new hidden state:

$$h_t = \tanh(C_t) \otimes o_t$$

useful info from new cell state becomes new hidden state

- g) LSTM forward pass

$$Z_t = \begin{bmatrix} \tilde{C}_t \\ f_t \\ i_t \\ o_t \end{bmatrix}$$

$$= \begin{bmatrix} W_c & U_c \\ W_f & U_f \\ W_i & U_i \\ W_o & U_o \end{bmatrix} \cdot \begin{bmatrix} x_t \\ h_{t-1} \end{bmatrix} + \begin{bmatrix} b_c \\ b_f \\ b_i \\ b_o \end{bmatrix}$$

$$= W I_t + b$$

$$h_t = \tanh(C_t) \otimes o_t$$

$$= \tanh(C_t) \otimes Z_o$$

$$C_t = C_{t-1} \otimes f_t \oplus i_t \otimes \tilde{C}_t$$

$$= C_{t-1} \otimes Z_f \oplus Z_i \otimes Z_{\tilde{C}}$$

GRU - 2 gates and 1 state

a) ~~hidden state~~

$$\tilde{h}_t$$

- A) update state - how much of past info need to be considered to next time step

$$Z_t = \sigma(W_z x_t + U_z h_{t-1})$$

- b) how much of past info to forget - Reset gate

$$r_t = \sigma(W_r x_t + U_r h_{t-1})$$

how in LSTM vanishing gradient does happen !!

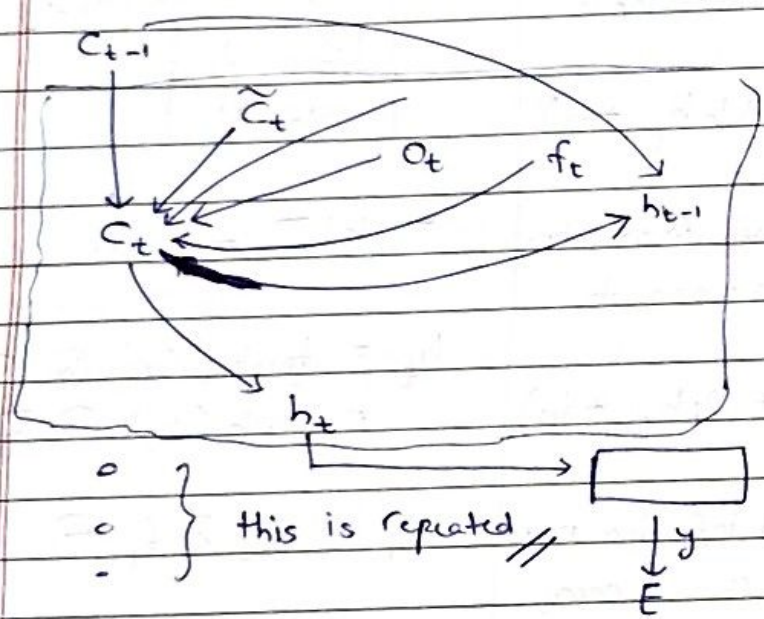
Date: 12 Mar, 26
(Day 18)

$$\frac{\partial E}{\partial w} = \frac{\partial E}{\partial y} * \frac{\partial y}{\partial h_t} * \left[\frac{\partial h_t}{\partial h_{t-1}} \dots \frac{\partial h_1}{\partial w} \right]$$

$\alpha \ll 1$
↑
vanishing grad
if α huge
↑
explanation
↑
As
Can be handled

→ Gates help in selecting info/
only some time frames

Can be applied in both fwd / backward



information
flow in
forward pass

$a \rightarrow b$
b is dependent
on a

→ backprop req for hidden and cell state both
↑
 h_t C_t

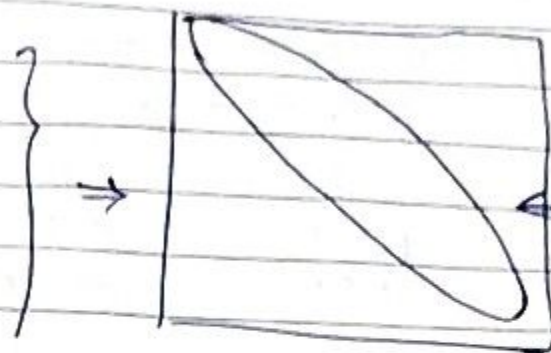
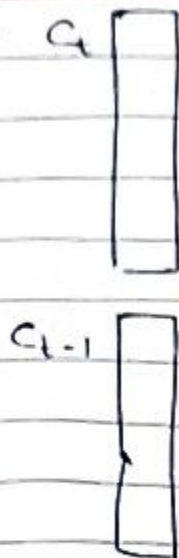
if C_t vanishes $\Rightarrow C_t = [C_{t-1} h_t] + [\tilde{C}_t i_t]$

both of these should
also vanishes ($= 0$)

if $C_t \neq 0$ and $(\tilde{C}_t i_t = 0)$ then we need to
ensure $C_{t-1} h_t \neq 0$

→ i.e. $\frac{\partial E}{\partial w} = \frac{\partial E}{\partial y} \frac{\partial y}{\partial C_t} \left[\frac{\partial C_t}{\partial C_{t-1}} \frac{\partial C_{t-1}}{\partial C_{t-2}} \dots \right] \frac{\partial C}{\partial w}$

(proof)* $\hookrightarrow \neq 0$

NOTE: G_t we get a
diagonal
matrix

$$* \text{ proof : } \frac{\partial E}{\partial w} = \frac{\partial E}{\partial y} \frac{\partial y}{\partial c_t} \begin{pmatrix} \frac{\partial c_t}{\partial c_{t-1}} & \frac{\partial c_{t-1}}{\partial c_{t-2}} & \dots \end{pmatrix} \frac{\partial c_0}{\partial w}$$

$$\begin{matrix} \uparrow & \uparrow & * \\ (f_t \cdot f_{t-1} \cdot f_{t-2} \dots) = c \end{matrix}$$

$$(f_t \dots f_0) = 0 \text{ only if :}$$

- (i) info not passed in fwd layer also
- (ii) (if passed in fwd pass, then req in back pass as well!)

- if exploding happens $\rightarrow$ we clip!
- similarly for GRU $\rightarrow$ we can expand the equations and find that vanishing gradient is handled! $\leftarrow$ (check it out)

(GRU contd)

c) Internal Information (current memory content)

$$\tilde{h}_t = \tanh(W_h \cdot x_t + r_t \otimes U_h \cdot h_{t-1})$$

↓
present
input

↓
useful information
from prev hidden state

d) hidden state (h_t)

$$h_t = (1 - z_t) \otimes h_{t-1} \oplus z_t \otimes \tilde{h}_t$$

includes info.
from the previous
hidden state which were
forgotten in internal info

↓
useful information
from $\tilde{h}_t$

Matrix $W = \begin{bmatrix} W_z & U_z \\ W_r & U_r \end{bmatrix}$ $I = \begin{bmatrix} x_t \\ h_{t-1} \end{bmatrix}$

$$z'_t = \begin{bmatrix} z_t \\ r_t \end{bmatrix} = \begin{bmatrix} W_z & U_z \\ W_r & U_r \end{bmatrix} \begin{bmatrix} x_t \\ h_{t-1} \end{bmatrix}$$

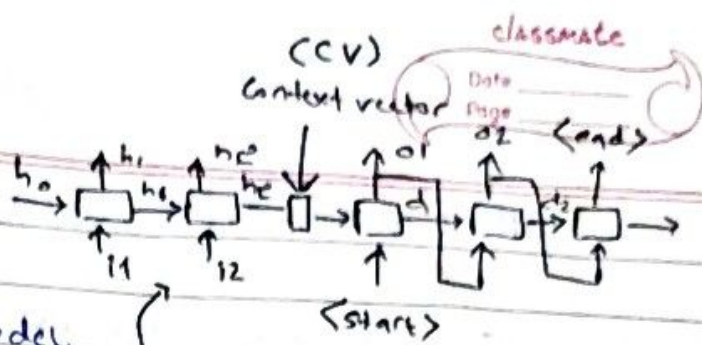
$$z'_t = W I_t$$

$$h_t = (1 - z'_z) \otimes h_{t-1} \oplus z'_z \otimes \tilde{h}_t$$

Day 19 (18/03/26)

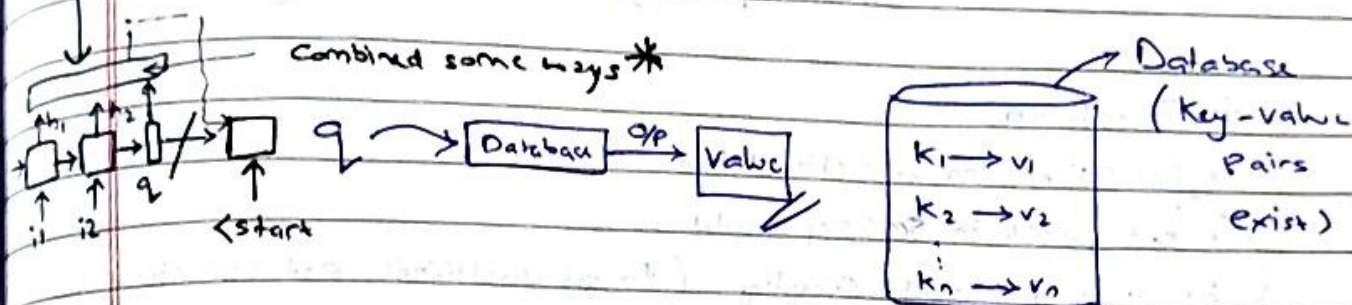
Seq2Seq Model

- one kind of generative model
- problem with encoder-decoder - output generated from the prev output only!
(Importance of which i/p word is not considered)



Attention

- for outputting 1 word, it gives wt to each one of the i/p word and picks the max one



NOTE: in diagram: red block is the query

* keys are the hidden op - h_1, h_2, h_3

S0: compute similarity / (sort of) alignment score

for each h_j and q

S1: apply Softmax to normalized $e(s_{ij}, h_j)$

it $\leftarrow$ attention weights

$= e_{ij}$

S2: get $CV = w_1 h_1 + w_2 h_2 + \dots + w_n h_n$

$\Rightarrow$ weight of h_1 given query q

instead of giving h_3 / intermediate representation, give the context vector which is the weighted sum of all input

ex. $f \rightarrow$ FFNN

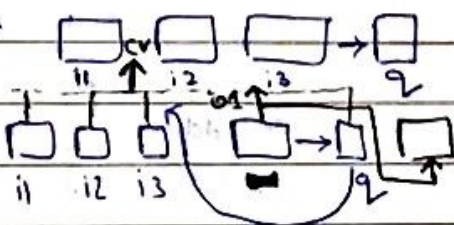
$e_{ij} = \tanh(w^T x)$

$x = [h_1, h_2, \dots, h_n, s_{i-1}]$

So basically for each o/p, get the output of prev hidden layer (or lka) $\rightarrow$ for first one

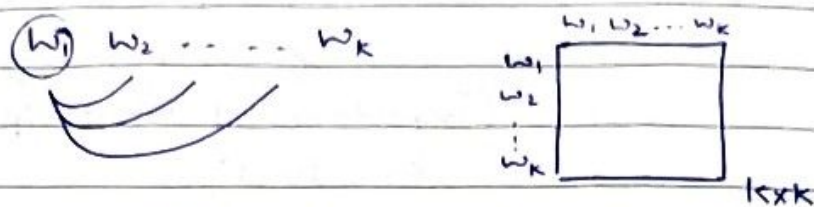
$$A(q, k, v) = \frac{\sum_j \exp(e_{qk_j}) v_j}{\sum_j \exp(e_{qk_j})}$$

for next one:



Self Attention - used by transformers

for one word generation which other words in the i/p influence?
i.e.



⇒ how to compute the influence?

{ Query becomes row value
Key is all column values
then compute (q, k) attention

Transformer

• Parallelism can be achieved here

• not possible in Seq2Seq model

by using positional encoding (to get positional and sequential info)

to identify position in

embedding - one hot vector

Input

glove

word2vec

masked attention - we know all i/p, but not $(t+1)$ outputs

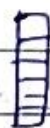
So at t we mask the future values

Positional Encoding

• Sine and Cos function

$w_1, w_2, w_3, \dots, w_n$

position
of w_i

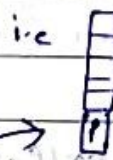


+

embedding
of w_i



M1: Add 1 more block in
embedding as Index i



⇒ Since discrete values, so it might create
pb during backprop i.e not smooth

⇒ it should be bounded and deterministic

↑
i.e. x if defined pos 1

then in all cases x should be pos 1

⇒ relative distance should also be maintained
discrete values so not smooth transition

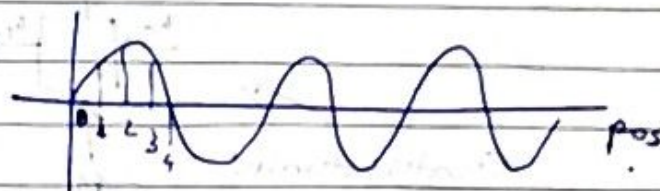
to satisfy these
why sin and cos?
⇒ Why not binary
encoder?
again

⇒ one sin wave - period of repetition is short

gets splitted i/p, pass
to each SA,

Multi Head Attention
- multiple self
attention but
different perspectives

Combine
results



Q and K from i/p
value

for self attention in
transformer

$$Q \times K = S \text{ of } C$$

normalize

softmax

self attention

wt matrix

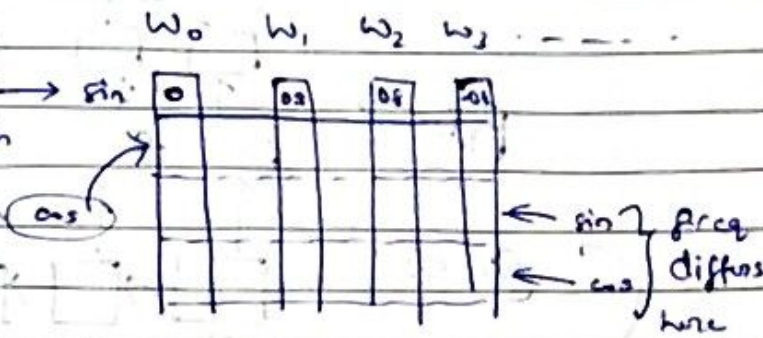
⊗

value

Transformed i/p

O/P of SA

only 1
not enough
so add



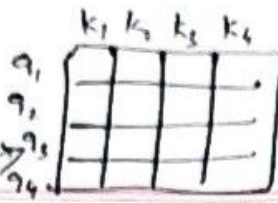
i.e. $\sin(x) \cos(x)$

• but still
after some
time, repetition
will happen

next $\sin(\frac{\pi}{2}) \cos(\frac{\pi}{2})$
 $\sin(\frac{\pi}{4}) \cos(\frac{\pi}{4})$

• so we should define sort of threshold
to handle it such that repetition not
possible

ex. sin - for even index } in paper
cos - for odd "



at $t=1$,
we would know
 k_2, k_3, k_4 so we
mask it!!

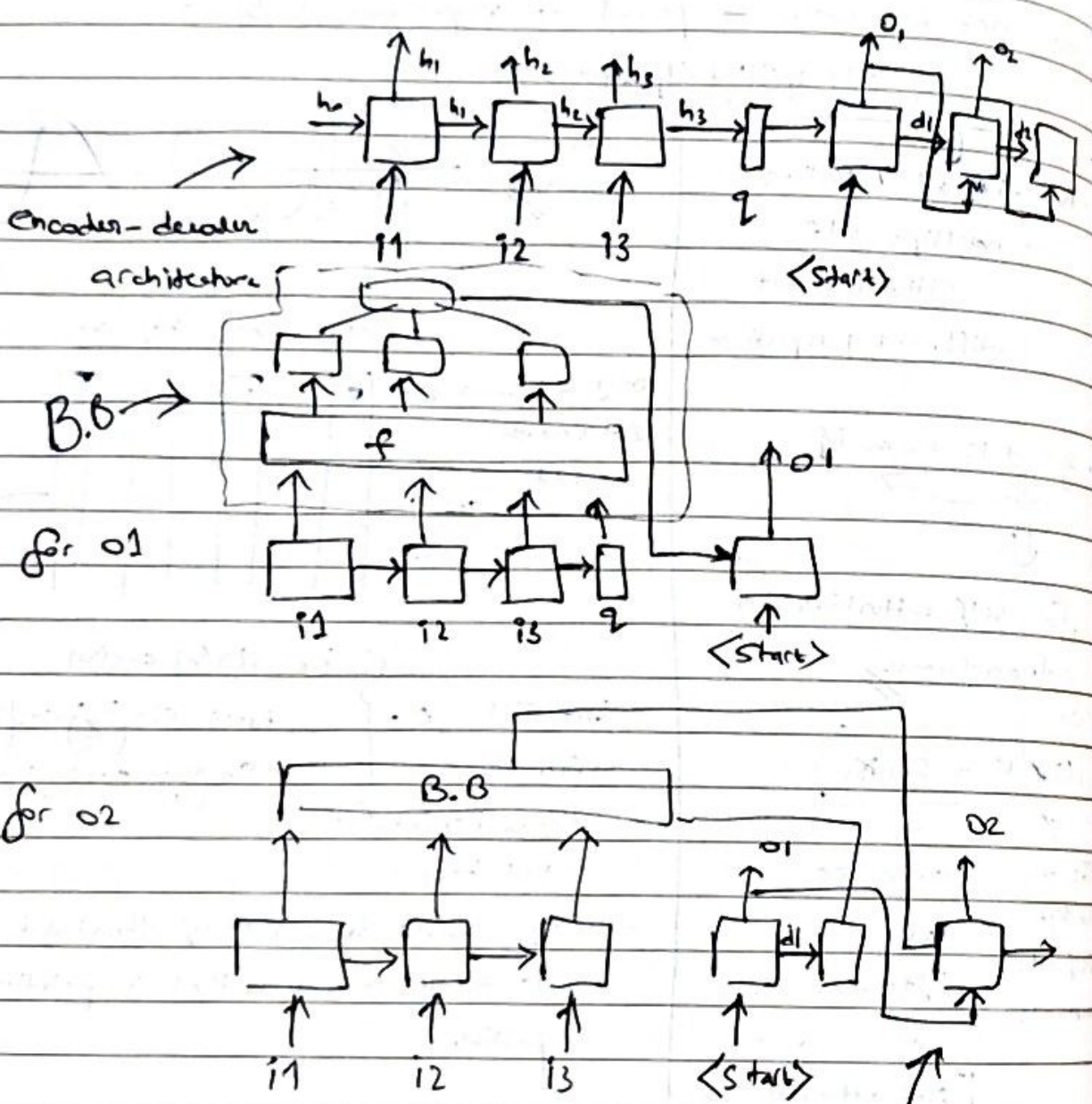
Decoder layer

↳ Masked M-H-A - avoid the effect of future

(has both key and query from same seq.)!

⇒ Whatever we want to produce is the query

that's why: decoder gets key and value from encoder



So here the hidden state part changes

Attention Types:

(i) Basic dot product: $e_i = q^T k_i$

(ii) Multiplicative attention:

$$e_i = q^T W k_i$$

weight matrices

(iii) Additive attention:

$$e_i = W_3^T \tanh(W_1 k_i + W_2 q)$$

weight
vector (1D)

for deeper
understanding
note: attention gathers
info, and FFNN
processes it //

Q. Why is the encoder has
attention, then ffnn?

Q. Why is there a classifier
at the end of Decoder?

Q. Why would we want a
stack of encoding or
decoding layer?

Q. Inputs are given one time
directly but what about
the outputs? how?

Q. What is decoder embeddings?

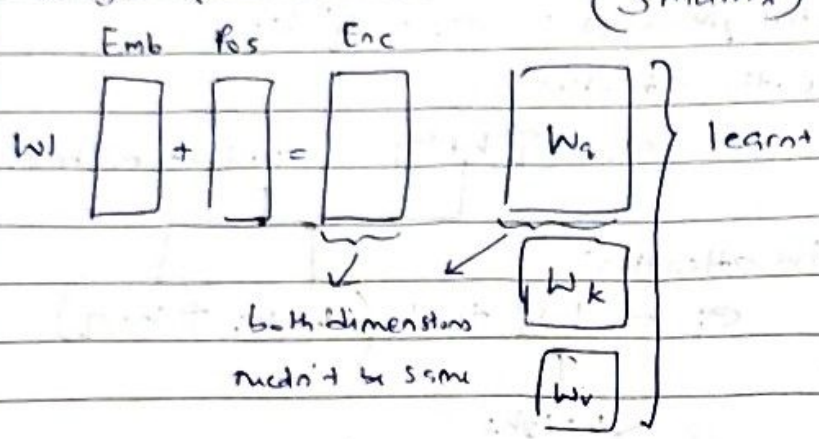
↑
(output)

Q. How is the seq-entail problem solved
here in transformer?

Encoding Layer

Day 20 (19/03/26)

(3 matrix)



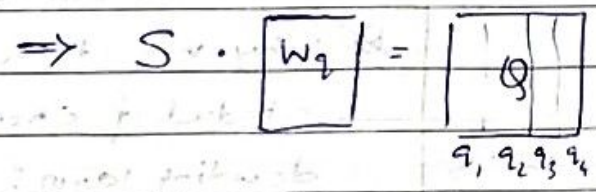
What are the learnable params?

→ in Q, K, V

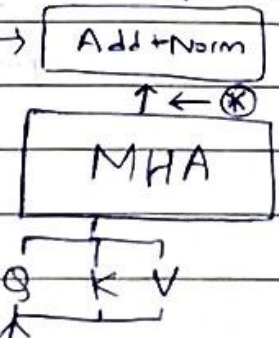
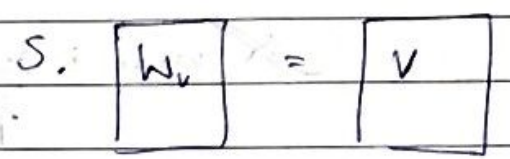
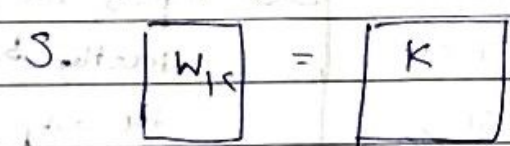
W_q, W_k, W_v

NOTE: $q_1 = W_1 * W_q$

$q_2 = W_2 * W_q$



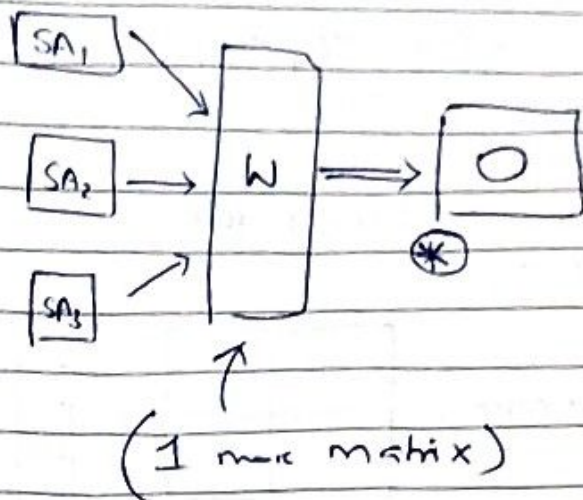
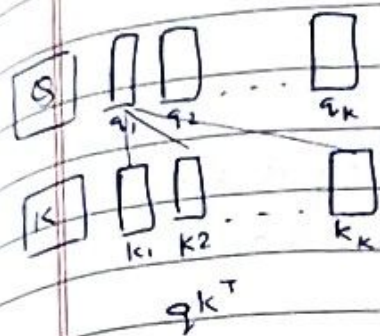
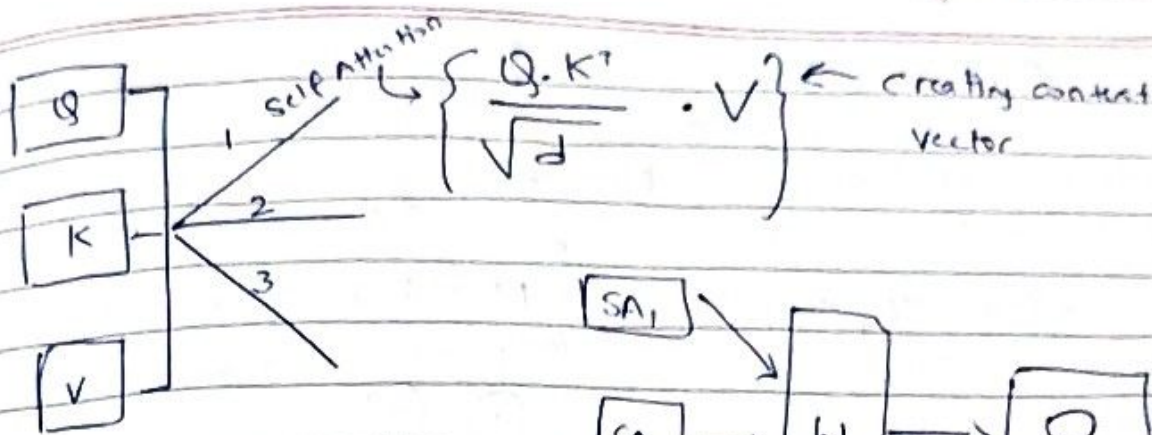
Sample all these into 8 parts (like 8 heads in MHA) → SHA that helps in seeing the samples from diff perspective



Pos → ⊕

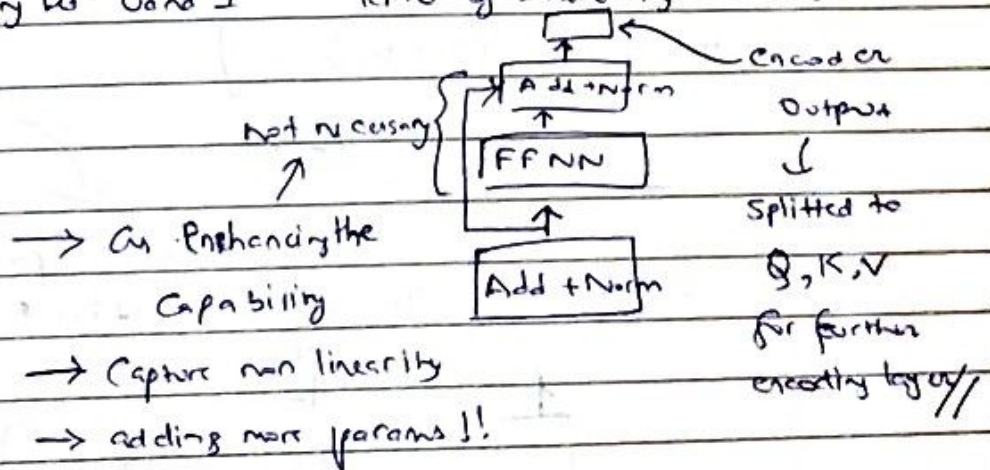
Emb



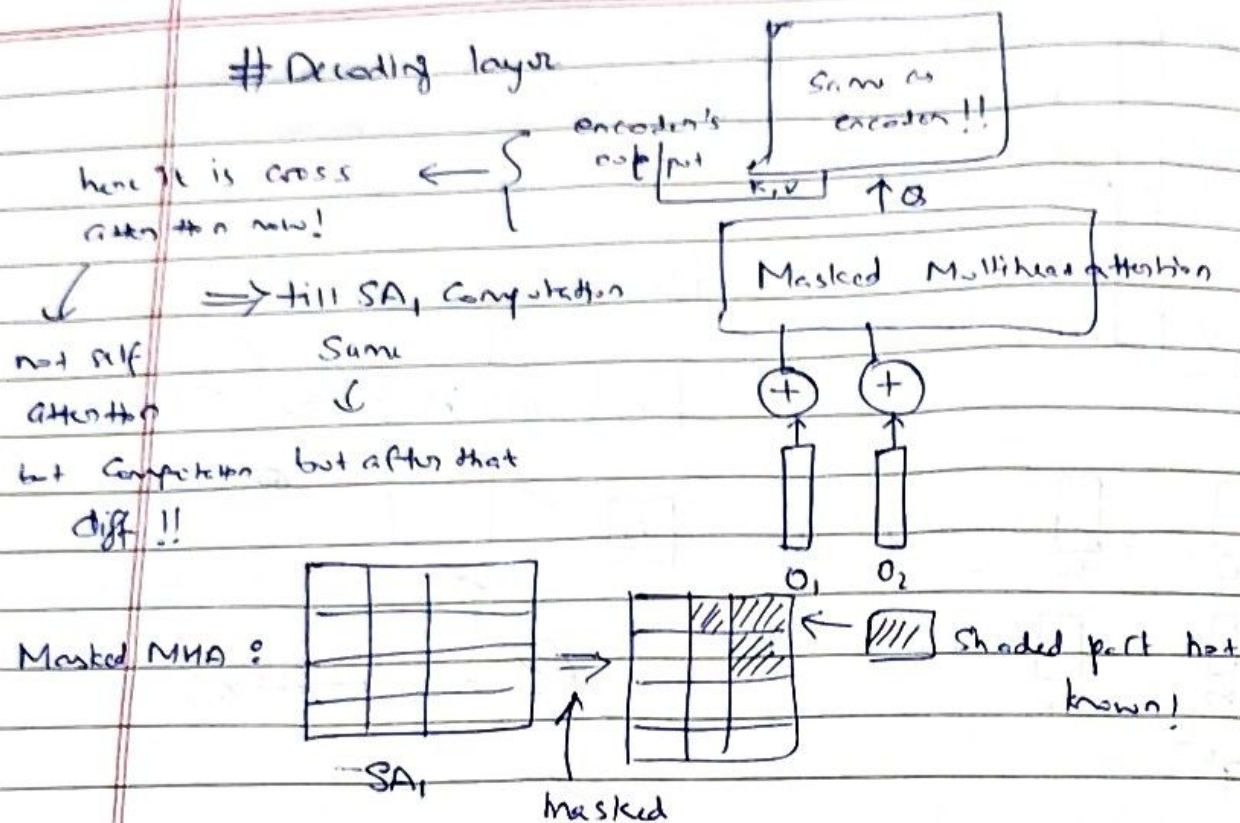


Add + Norm and residual ~~connection~~ connection

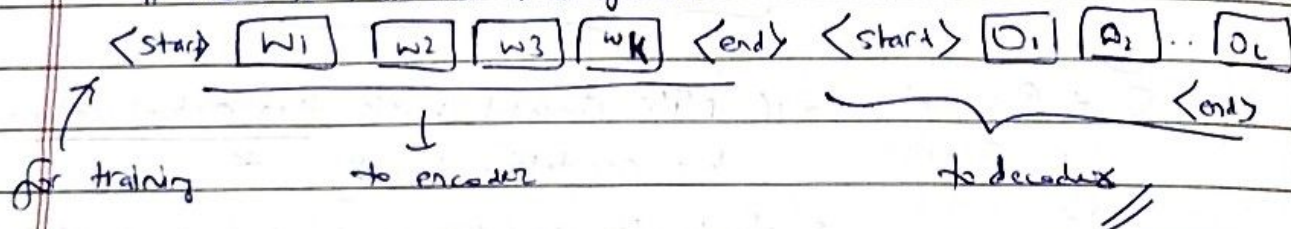
- i/p: $O + \text{Residual Connection}$
- if MHA doesn't work then actual values we are taking directly *
 - to handle vanishing gradient: sort of switch
 - it helps in back propagation
 - sort of adjusting the gradient
 - then apply layer normalisation on it
 - i.e. $\text{Norm}(O + \text{Res})$
 - Varying W^n and 1 - kind of smoothing the values



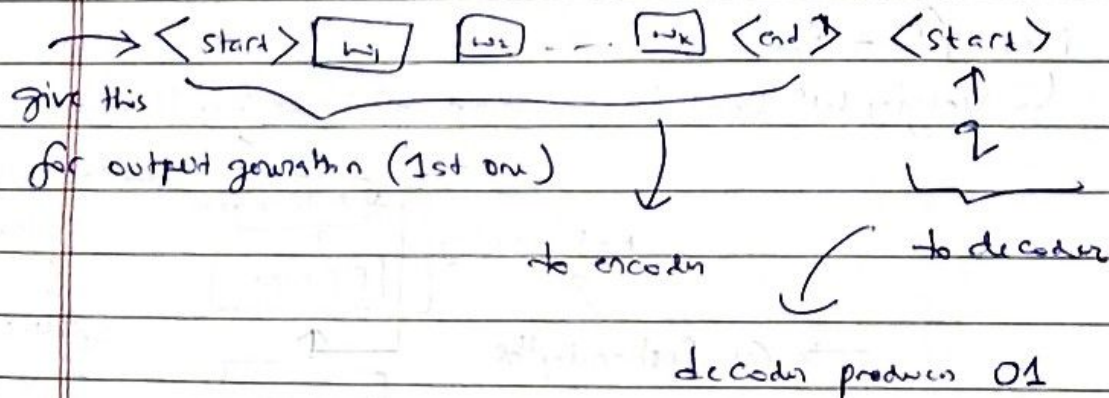
Decoding layer



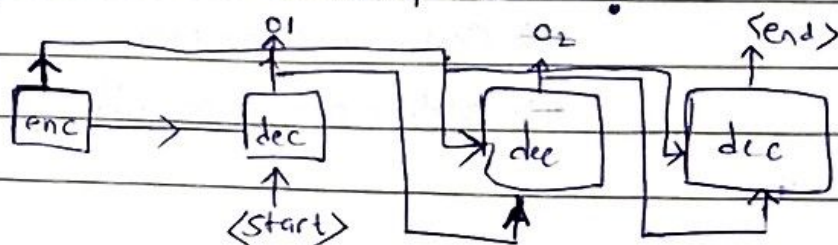
How to train the transformer



What are for inference?



now for o₂: It will be sequential!



BERT Embedding

→ contextual dependency cannot be obtained - in word2vec, GloVe etc

→ BERT solves this by using encoder part of transformer

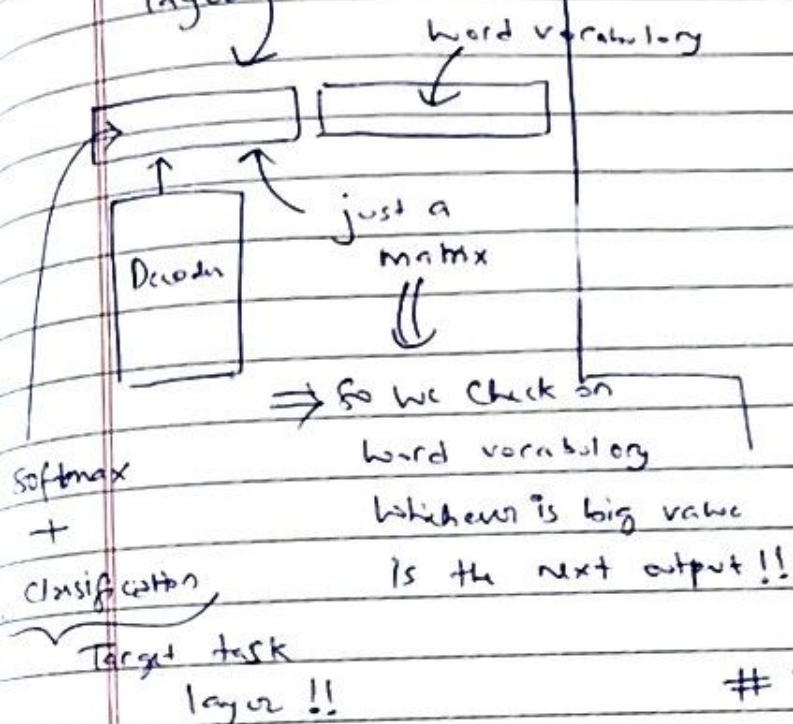
→ it will output diff embedding depending upon context for the same word

how to do BERT training? → (NLP)

→ Sort of next word prediction training can be done or masked language model *

→ This is multi tasking

After decoding,
we have a linear
layer



Learning Process

(i) Transfer learning

→ build a model from available data

(pretrained model)

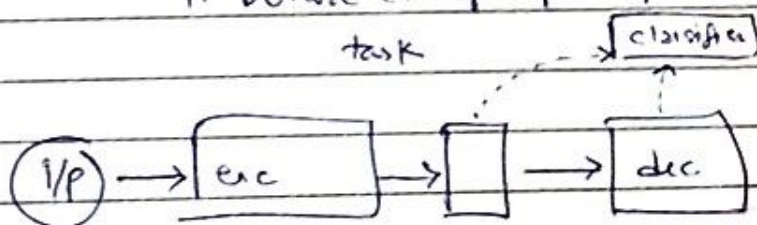
↓
(low resource) fine tuning on the base model

(ii) Multitasking

→ build a model to do multiple task

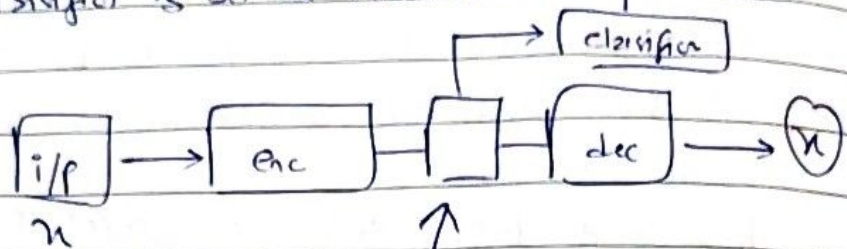
ex. NLP + Mask Language model can be done

Where classifier part depends on target task



Auto encoder focuses on i/p representation

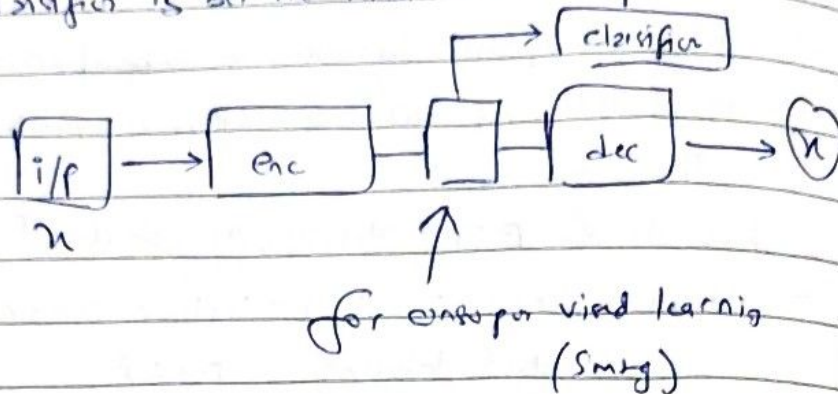
⇒ classifier is on the intermediate representation



↑
for supervised learning
(Smog)

Auto encoder focuses on i/p representation

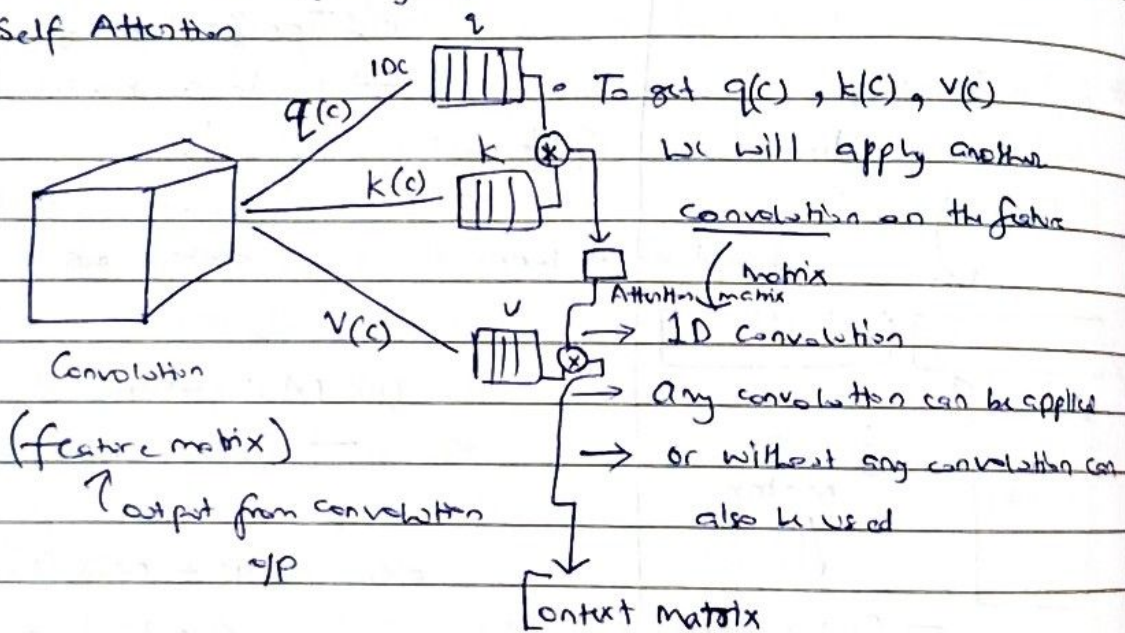
⇒ classifier is on the intermediate representation



Day 21 (24/03/26)

* Attention in CNN (4 types)

(i) Self Attention



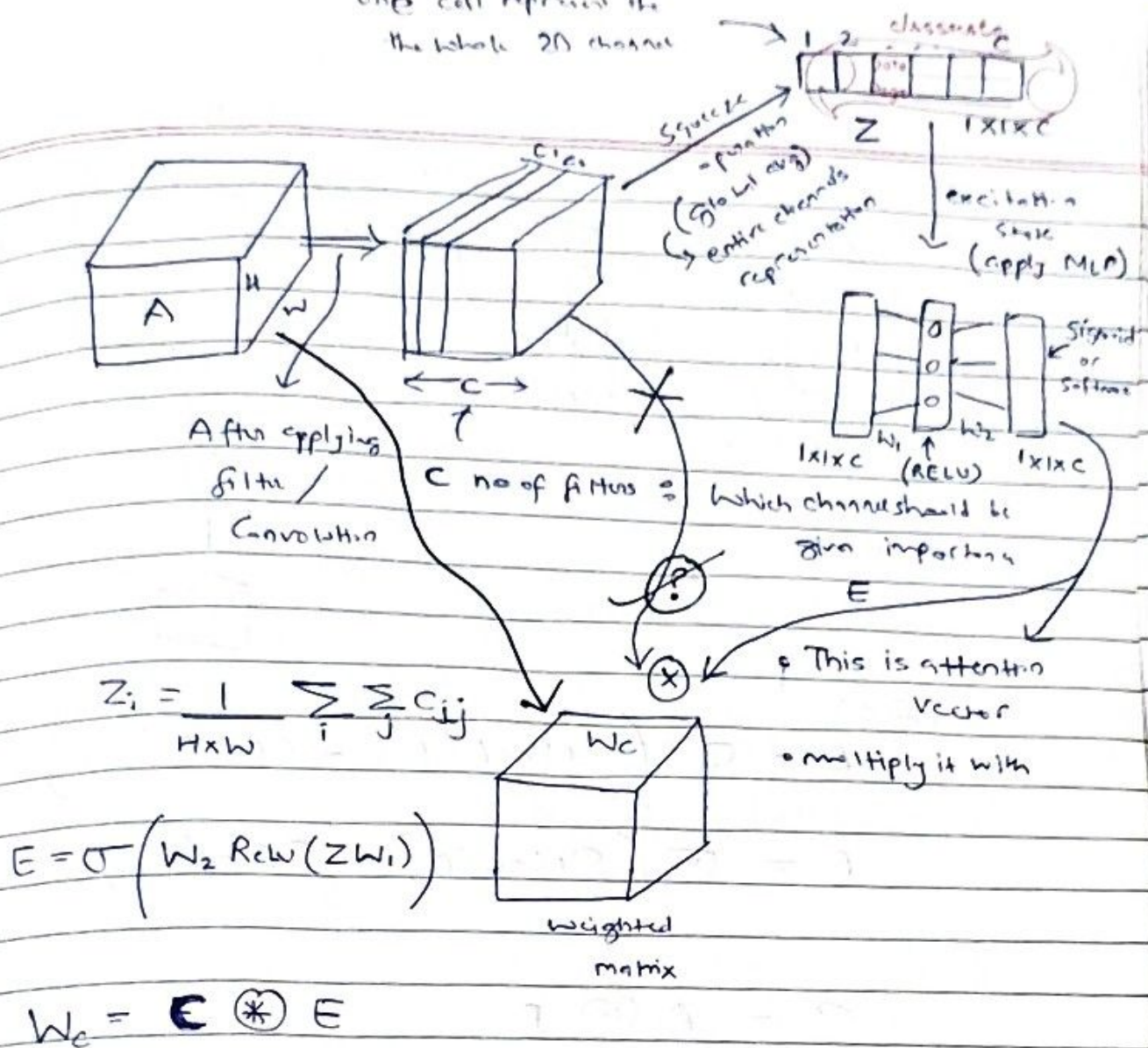
(ii) Squeeze and Excitation (Channel Attention)

→ Multiple filter applied on i/p

↓
Channels

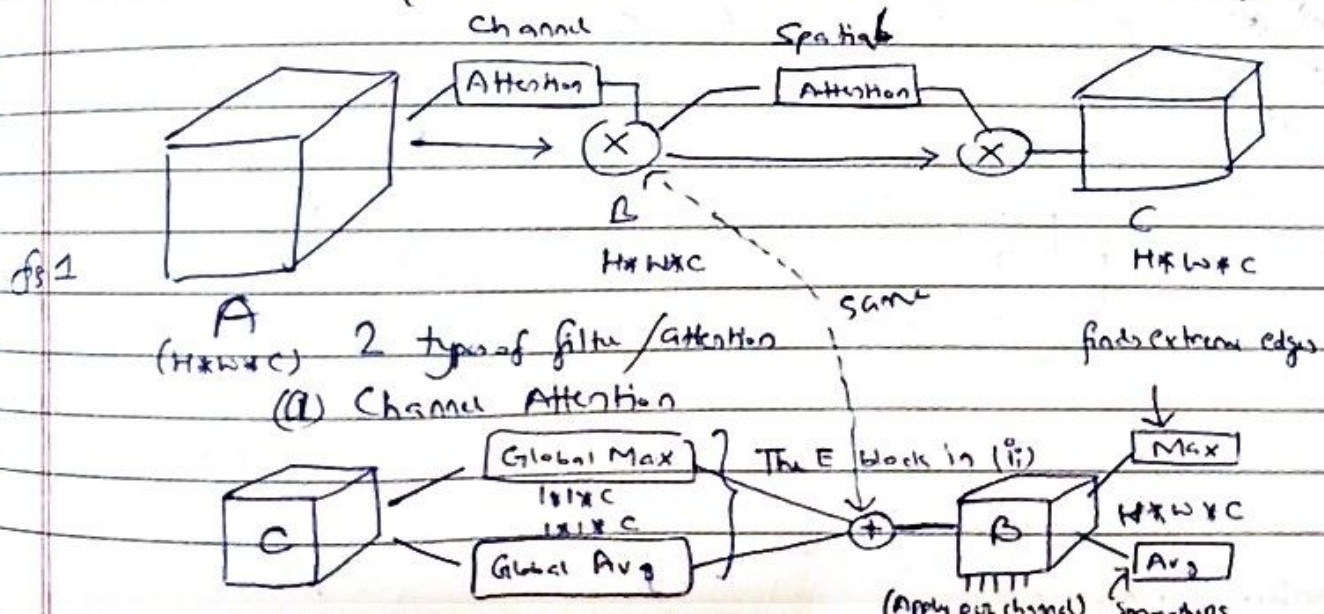
→ Which channel should be given more importance?

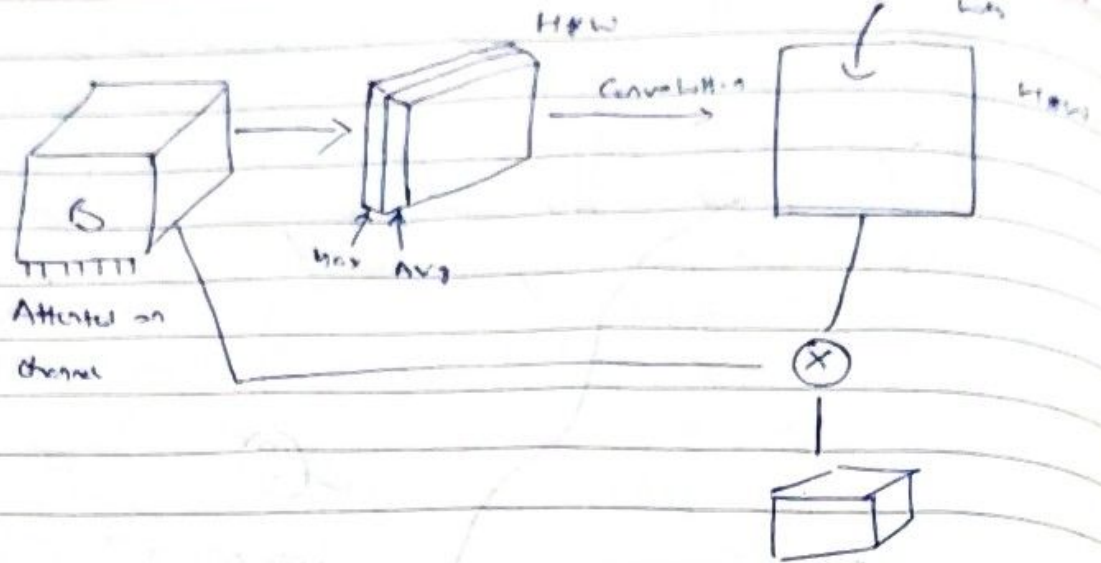
one cell represent the
the whole 2D channel



In Case of Spatial attention $\rightarrow$ In the channel which area
should be focused is
the Importance

(iii) CBAM (Convolutional Block Attention Module)





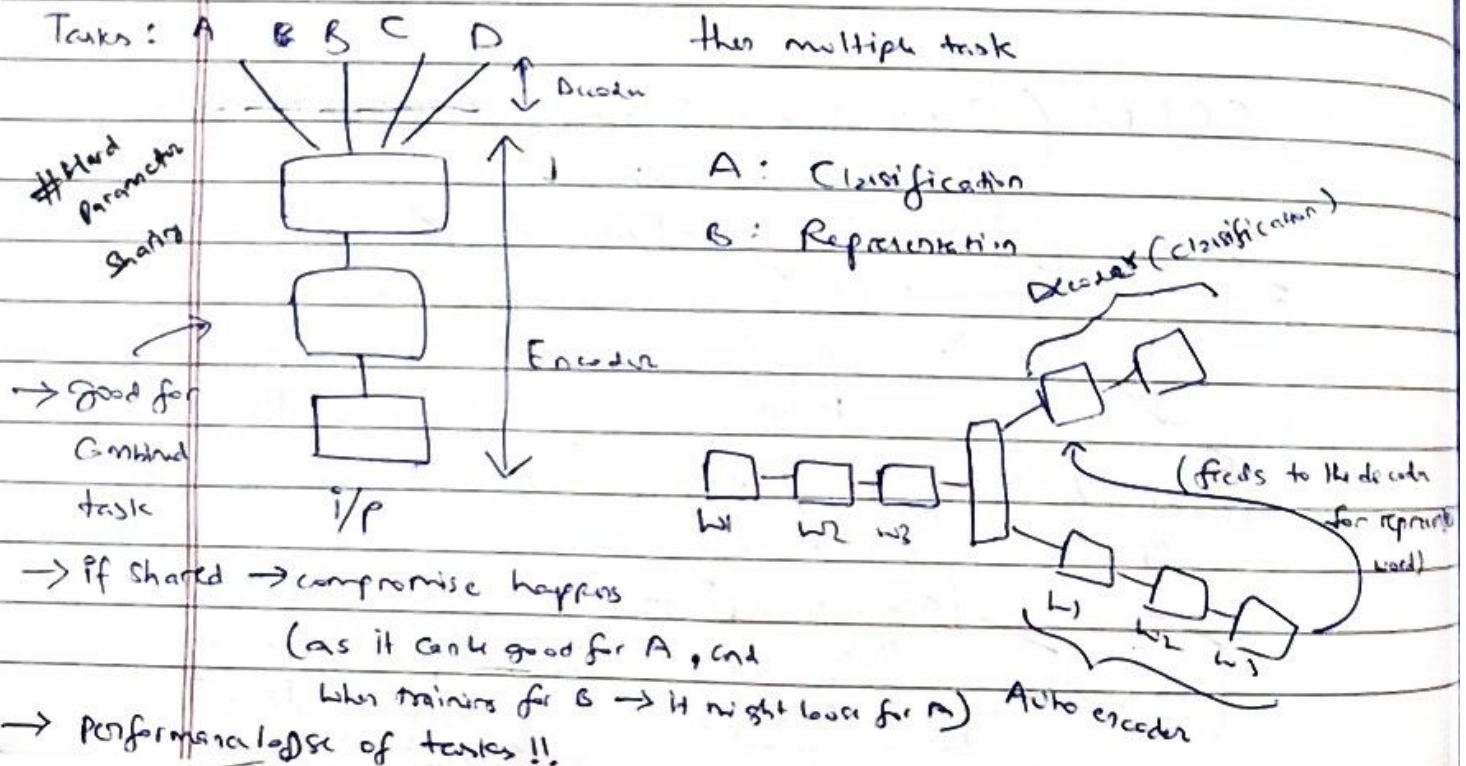
(Eq 1)

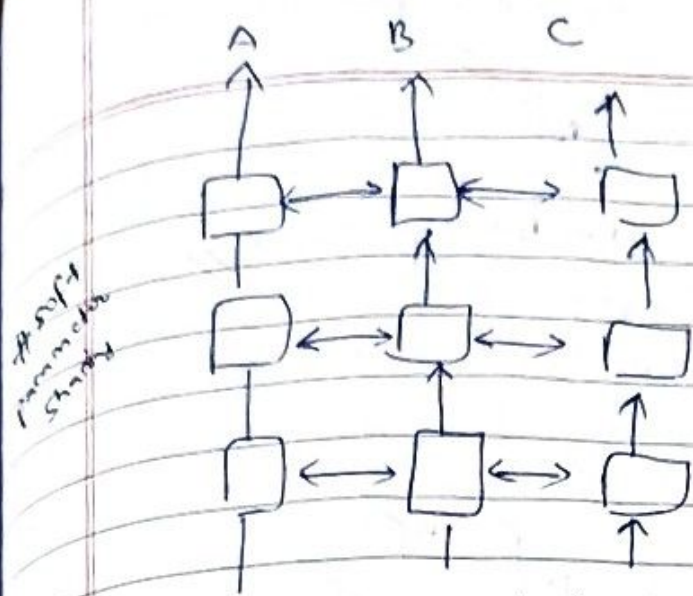
$$B = \sigma \left(C_{MaxA}(A) + C_{AvgA}(A) \right)$$

$$B = \sigma \left(S_{Max}(B) + S_{Avg}(B) \right)$$

$$C = A \otimes B$$

→ (most multi-modal have multi-tasking set up)
 * Multi-Tasking (shared networks)
 ↓

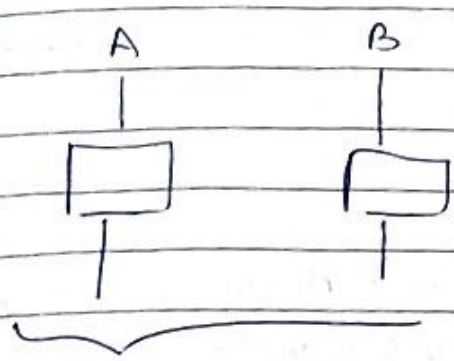




here info is shared
each task has a separate
setup/networks

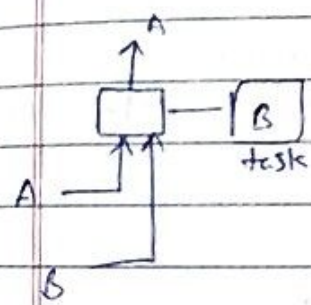
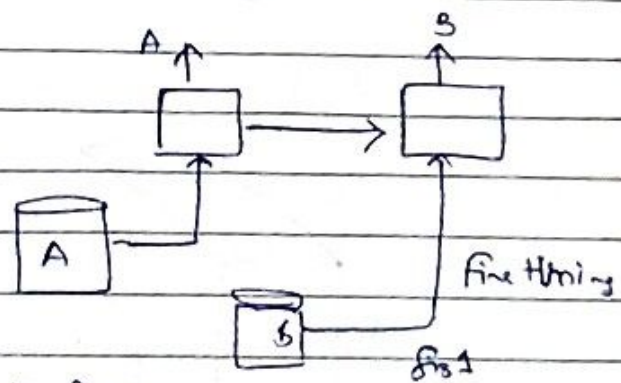
Partial
Parameter
Sharing

* Transfer Learning
→ good model for 1 task → transfer it for another task



→ but if for A large
dataset exists
and small dataset
for B

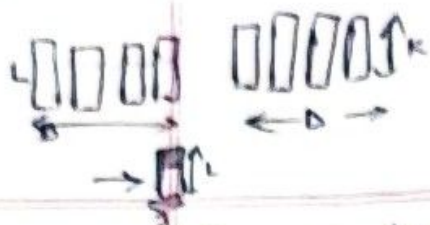
→ Separate models
→ req good dataset for
both A and B



in Zero shot don't fine tune.

← but directly train the model with A and B
and do task B

fewshot training (fig 1)



→ Group, Layer, 1x1
 keep the same dimensions but reduce parameters (in depth)
 check examples (!)

Day 22 (25/03/26)

1D, 2D and 3D convolutions

When we do convolutions, H depends on task which one we use
 how what are we computing
 1st cell of p matrix
 To which the filters determine!

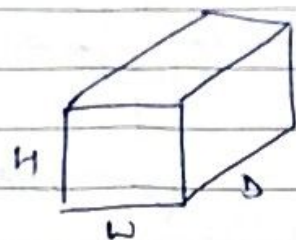
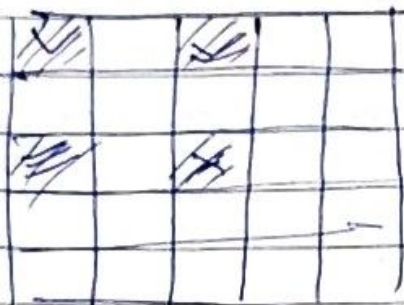
Dilated Convolution

filters determine!

a cell can be

computed using

1D values or 2D values



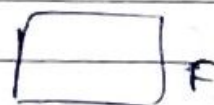
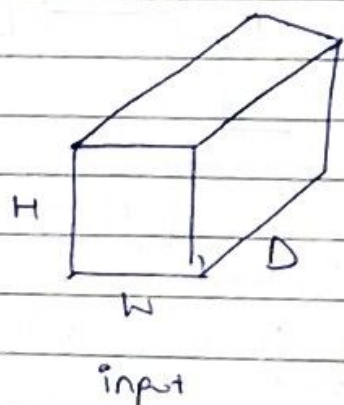
NOTE: 3x3 filter

→ 3x3xD here depth is considered by default

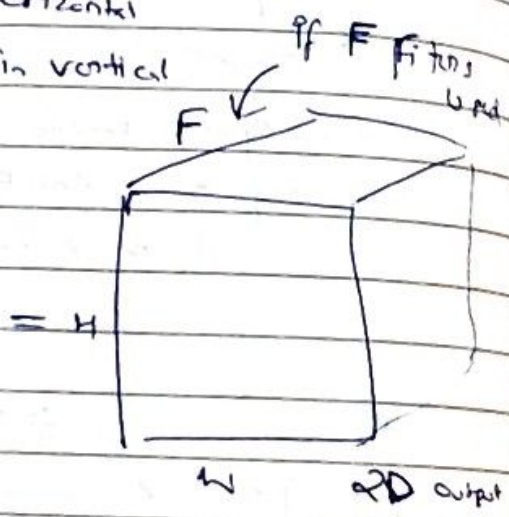
NOTE: 2D convolution in tensorflow

has dilation factor!

for example: 1x1 ← skip 1 in horizontal and skip 1 in vertical



3x3
 ↓ ↓
 H W



parameters: 3x3xD → depth

if F no. of filters ⇒ # = 3x3xDxF

↑ ↑ ↑ ↑
 H W depth filters

NOTE: Any conv can be applied on any dimensions

except $\rightarrow$ 1D $\rightarrow$ req (length $\times$ channel)

2D $\rightarrow$ (height, width, channels)

3D $\rightarrow$ (depth, height, width, channels)

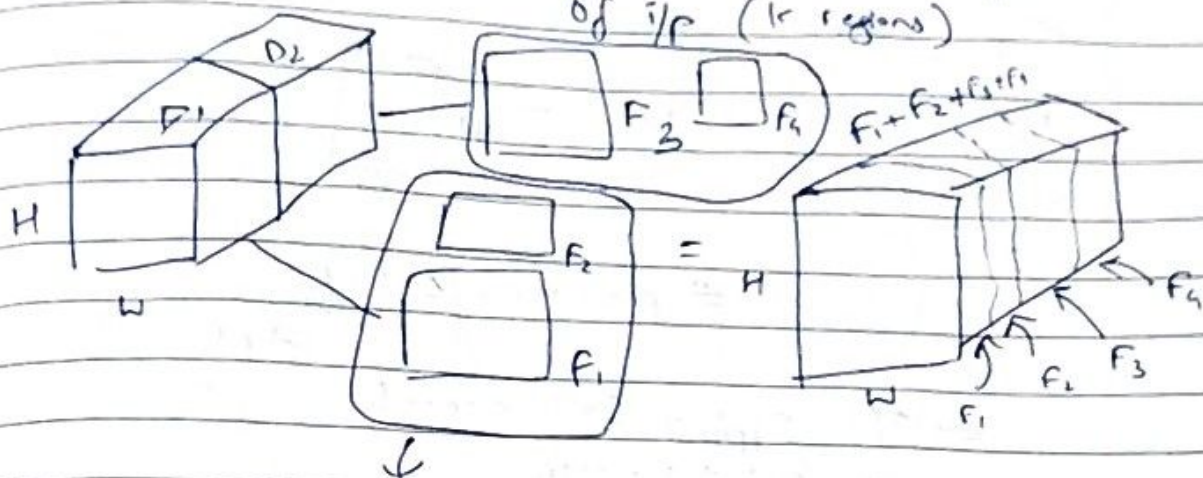
Associate

Out
Depth

i/p structure

Group Convolution

filter applied on different regions of i/p (K regions)



parameters = $3 \times 3 \times D_1 \times 2$

$\Rightarrow K \times K \times F \times (D_1 + D_2)$

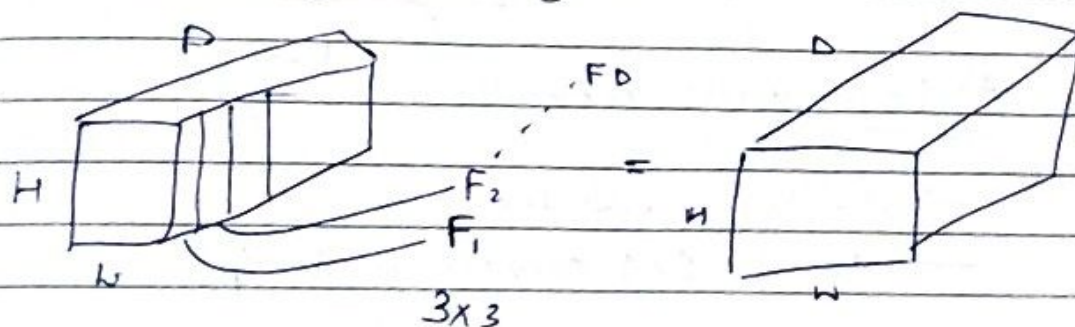
$+ 3 \times 3 \times D_2 \times 2$

$= (3 \times 3) (D_1 + D_2) \times 4 \times 2$

Split Convolution (or Layer Convolution) (or depth wise convolution)

Can we consider D no. of filters?

$\rightarrow$ So each layer has 1 filter



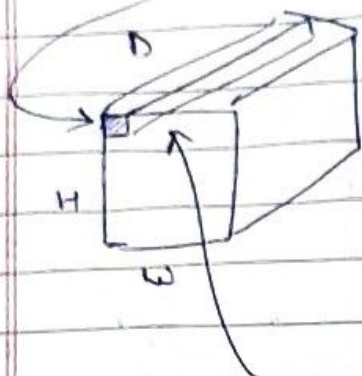
$\Rightarrow 3 \times 3 \times 1 \times D$

((depth no. of filters

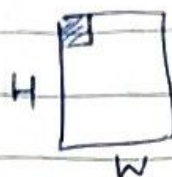
$(K \times K \times 1 \times F)$

(Point wise convolution)

1x1 Convolution



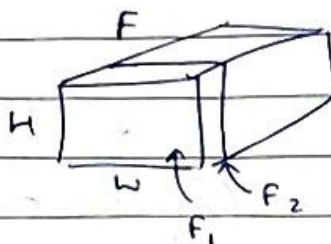
1x1
Con



parameters = $1 \times 1 \times D$
↑
depth

⇒ After applying 1x1 across D
we need to sum it up

If multiple features we want to extract then apply
F filters ⇒



parameters = $1 \times 1 \times D \times F$

Layer separable convolution ($K \times K \times D \times F$)

↑ as well !!

→ do layer convolution

→ then do 1x1 convolution on the o/p

($K \times K \times D$
+
 $1 \times 1 \times D \times F$)

Summing

2D → $K \times K \times F$ ← filters

Parameters: $K \times K \times D \times F$

grouping → $K \times K \times F \times (D_1 + D_2 \dots)$

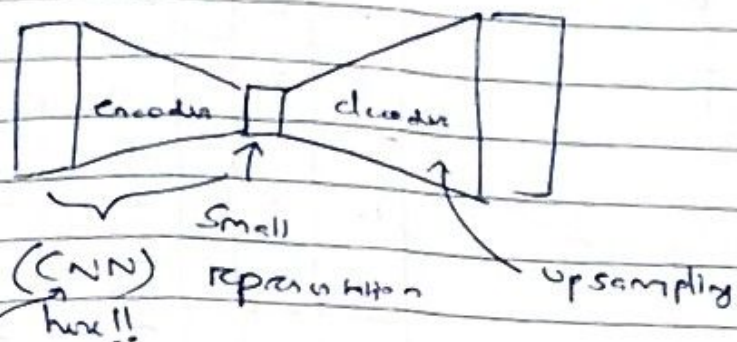
layer → $K \times K \times D$

1x1 (F) → $1 \times 1 \times D \times F$

layer-separable → $K \times K \times D + 1 \times 1 \times D \times F$

Encoder and Decoder in CNN

- In CNN $\rightarrow$ down sampling happens in encoder



leads to reduce in

dimension

(down sampling)

done by pooling layer

Q how to do upsampling?

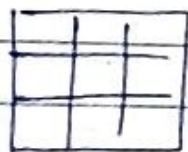
$\Rightarrow$ pooling layer should be unpooling to increase dimension

$\Rightarrow$ done by:

(a) nearest neighbour

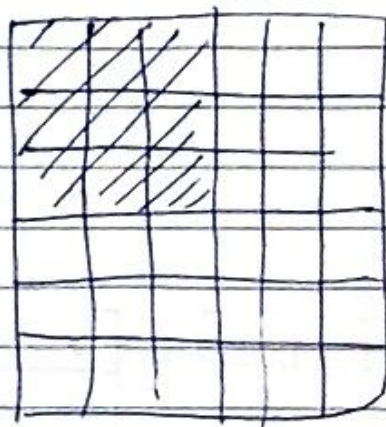


pooling



copy kernel
on the
pooling
layer

$\Rightarrow$



$\rightarrow$ kernel

$\rightarrow$ for upsampling we need kernel (also

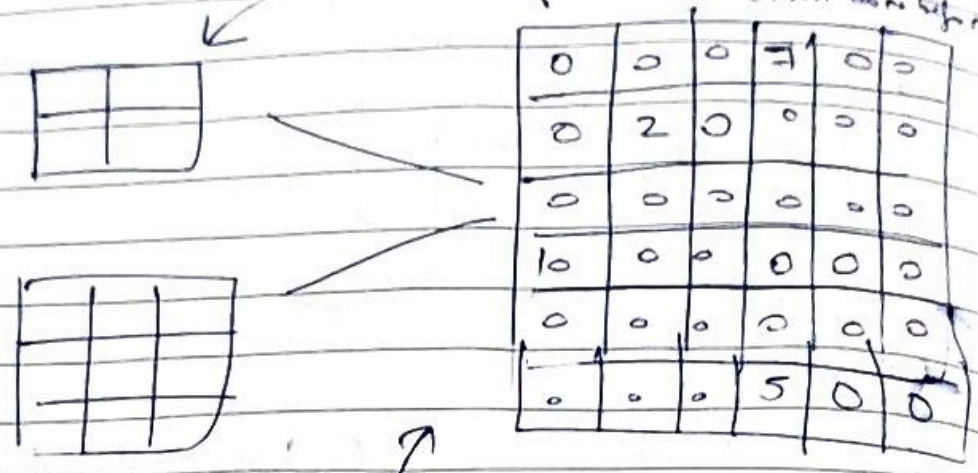
$\rightarrow$ but not learnable!!

$\uparrow$ disadvantage

$\rightarrow$ just defines the expansion cell (size may be?)

$\rightarrow$ in nearest neighbour, it the same values in just copy all the ~~same~~ cells

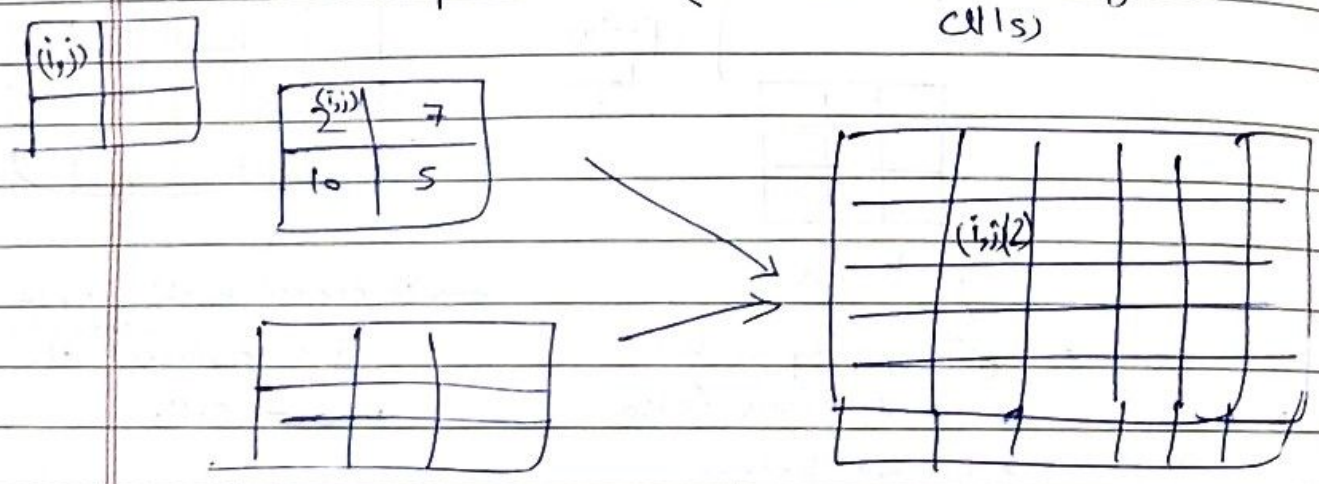
(b) Bed of Noll $\rightarrow$ (if we want to go back do the same avg/max pooling with ch less done before)



$\rightarrow$ select a cell in 3x3 block
 $\rightarrow$ fill it same as pooling cell rest all zeros

(c) Max unpooling

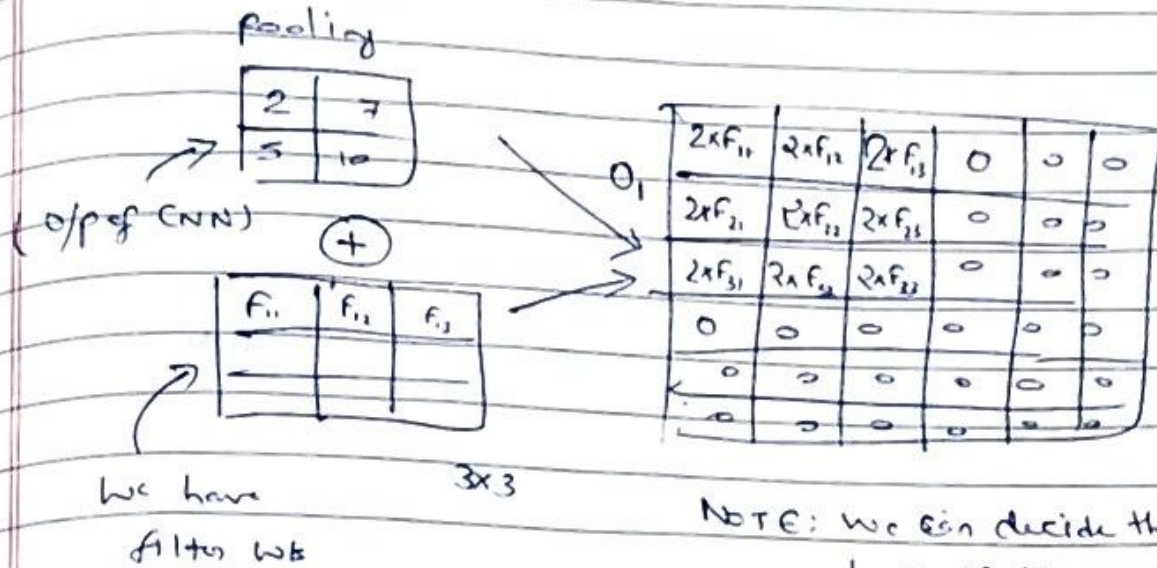
o Store the location from which it is pooled $\rightarrow$ (o in that location $\rightarrow$ fill the cells)



(Used in U-NET)

(d) Transpose Convolution

→ in all prev method the kernel is not learnable
but this one is:



O_2

0	$7f_{11}$	$7f_{12}$	$7f_{13}$	0	0
0	$7f_{21}$	$7f_{22}$	$7f_{23}$	0	0
0	$7f_{31}$	$7f_{32}$	$7f_{33}$	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0

O_3 O_4

similarly generate

$$\Rightarrow \text{final } O = O_1 + O_2 + O_3 + O_4$$